



Efficient Runge–Kutta solver for stochastic crack growth analysis

Wellison José de Santana Gomes^{a,*}, André Teófilo Beck^b, Alexandre Galiani Garmbis^c

^a Center for Optimization and Reliability in Engineering (CORE), Department of Civil Engineering, Federal University of Santa Catarina, Rua João Pio Duarte, 205, Córrego Grande, 88037-000, Florianópolis, SC, Brazil

^b Department of Structural Engineering, University of São Paulo, São Carlos, SP, Brazil

^c Petrobras, Rio de Janeiro, RJ, Brazil



ARTICLE INFO

Keywords:

Probabilistic fracture mechanics
Stochastic fracture mechanics
Stochastic crack growth analysis
Runge–Kutta
Initial value problem

ABSTRACT

Stochastic crack growth analysis by simulation may easily require a significant amount of computational effort. The Initial Value ordinary differential equations appearing in crack propagation problems may be solved by the Runge–Kutta (RK) family of solvers. However, crack propagation considering uncertainties imposes additional challenges for RK solvers: possibility of vectorized and/or parallelized computation of sample realizations; and adaptation of time discretization for each sample, to handle changes and discontinuities in the derivatives of crack growth curves, caused by changes in loads and in crack propagation rates. Existing adaptive RK methods, with change in time step length during crack growth computation, struggle with these challenges. In this setting, the present paper proposes a modified adaptive RK method which allows for simultaneous crack growth computations with different time-step discretizations, and aims at efficiently dealing with cases where discontinuities are present in the derivative of the crack growth curve. Application of the proposed method to three crack propagation examples, including one related to weld flaws in pipes, indicates that it is as accurate as other adaptive methods from the literature, while requiring only a fraction of the computational time.

1. Introduction

The so-called Initial Value Problem (IVP) is basically an Ordinary Differential Equation (ODE) together with initial condition(s) [1], while ODEs are equations which describe how quantities change. IVPs are found in many areas [2,3], including structural engineering [4–7]. Several methods are available in the literature to deal with IVPs, including for example the Taylor series method, linear multistep methods and Runge–Kutta methods [1].

Runge–Kutta (RK) solvers are a family of methods which have been successfully applied in the solution of lots of IVPs in the last decades [8–11]. The original RK method was proposed many years ago [12,13], but has been widely studied, applied, and significantly improved ever since [10,14–18].

In the field of fracture mechanics, crack propagation problems may be defined by a set of ODEs with given initial values. For example, Paris' crack propagation law and the initial values of crack depth and length. Solution of these problems is obtained by integrating the ODEs over the load cycles, considering the initial values, to determine the respective final values. These problems may be seen as IVPs and RK methods are effective and widely used options to solve them [19].

Existing studies on the application of RK methods to fracture mechanics include, for example: [20], where a modified RK solver for fatigue crack growth analysis is proposed, accelerating the integration

process but overestimating the crack growth; [21], where an adaptive RK method is employed in the risk assessment of airframe digital twin structures; among many others [22–26].

However, if thousands or millions crack propagation realizations are required to solve a given problem, as in stochastic crack growth analysis, some challenges arise. First, performing the computations one by one is clearly inefficient. Hence, it is highly recommended to perform as many simultaneous simulations as possible, in a vectorized and/or parallelized way, to reduce the computational time. Second, it is common to have loads and/or crack propagation rates which significantly change over time [27], especially in practical applications, and this leads to significant changes and even discontinuities in the derivative of the crack growth curve. This means that the solution is continuous, but not differentiable. As a consequence, different discretizations, with different mesh points over time, are required for each sample realization, in order to achieve the desired accuracy. As stated by Dieci and Lopez [28], a numerical method may become either inaccurate or inefficient, or both, in regions where discontinuities of the solution or its derivatives occur.

In the context of differential systems with discontinuous right hand side, two types of methods are available [28]: event driven methods, which detect the discontinuities by using an event function; and time-stepping methods, which detect the discontinuities by controlling the local errors.

* Corresponding author.

E-mail addresses: wellison.gomes@ufsc.br (W.J.d.S. Gomes), atbeck@sc.usp.br (A.T. Beck), alexandregaliani@petrobras.com.br (A.G. Garmbis).

An event function may be employed to verify when load changes have a significant impact on the crack propagation behavior. Continuous extensions and interpolants for RK formulas [29–32] can be used to accelerate the event detection. Unfortunately, many numerical methods of this kind are applicable only if certain conditions, such as the transversality condition (see [28]), are satisfied. The transversality condition implies that each discontinuity point is a transition point, that is, the event function changes sign at the discontinuity point [28,33]. When dealing with stochastic crack growth analysis, it is difficult to determine an event function which covers all significant events. It is also very difficult to have such conditions satisfied. Furthermore, event location comes with some added computational cost, usually with the application of root finding methods, which the authors wish to avoid here.

A review on the numerical solution of discontinuous IVPs by RK codes which make use of the transversality condition may be found in [34]. However, all presented methods use searches for the discontinuity in a number of subintervals for each step, in each simulation. This would be inefficient when performing lots of simulations where just a few of them present significant discontinuities, which is the case of stochastic crack growth analysis, of concern in the present paper.

On the other hand, time-stepping methods do not require an event function [28], but change the step size adaptively, based on an estimate of the error. Yet, adaptive RK methods available in the literature also struggle with the above-mentioned challenges. This happens because these methods are usually either very good at performing one simulation at a time, or they are able to deal with many simulations simultaneously, but are not very good at dealing with the discontinuities.

For these reasons, the present paper proposes a modified adaptive RK method which allows for simultaneous crack growth simulations with different time-step discretizations. The method aims at efficiently dealing with stochastic crack growth analysis by simulation, and focuses on cases where discontinuities and/or significant changes are present in the derivative of the crack growth curve.

The remainder of this paper is organized as follows. In Section 2, the crack growth initial value problem is introduced and the classical Runge–Kutta method is presented. In Section 3, brief descriptions and discussions about adaptive RK methods are given, and a modified adaptive RK method is proposed and briefly compared to the original adaptive method. Section 3 also describes the stochastic load simulation method employed in this paper. Section 4 presents three crack propagation examples, where the proposed method is compared to two methods from the literature. Finally, some conclusions are drawn in Section 5.

2. Crack growth initial value problem and the classical Runge–Kutta method

According to Stoer and Bulirsch [35], many problems in applied mathematics consist of seeking a differentiable function $y = y(x)$ of one real variable x , whose derivative $y'(x)$ satisfies an equation of the form:

$$y' = f(x, y(x)), \quad (1)$$

which is called an ordinary differential equation.

In general, there are infinite solutions to this problem. However, by defining initial conditions of the form $y(x_0) = y_0$ at a given point x_0 , one has an Initial Value Problem [35], which may have a unique solution. In some cases, it is possible to seek a solution analytically, but the present paper focuses on numerical solutions in the form of sequences of numbers approximating $y(x)$ at discrete sequences of values of x . Since x is, in fact, continuous, the process of choosing the discrete values of x where to compute $y(x)$, the so-called mesh points, is called discretization.

Crack growth problems are IVPs [36] where the initial dimensions of a crack are known and, by considering fatigue crack growth rate

models (see, for example, [37–41]), it is possible to compute the size of the crack at a specific time. Basically, these models try to establish how much the dimensions change when the crack has a given configuration and load cycles are applied to the structural element which contains the crack. The crack dimensions at the end of the lifetime of a given structural element are commonly of particular interest.

Taking into account the crack depth, a , for example, and the number of load cycles, N , the differential equation which relates these quantities may be given, for example, by the Paris law [39,42]:

$$\frac{da}{dN} = C [\Delta K(N, a(N))]^m, \text{ for } \Delta K(N, a(N)) > \Delta K_{th} \quad (2)$$

where $\frac{da}{dN}$ is the differential crack growth per cycle, ΔK is the range of stress-intensity factor within the cycle, C and m are material constants that are determined experimentally, and ΔK_{th} is a fatigue threshold. For $\Delta K < \Delta K_{th}$ it is assumed that no growth occurs, as observed experimentally [39]. The fatigue crack growth rate in metals can usually be described by the empirical relationship given in Eq. (2) [39].

Stress-intensity factors, K , may be determined by means of numerical methods, such as the finite element method. Results obtained numerically are directly employed in stochastic crack growth analyses, summarized in tables or employed in the construction of approximations [43–45]. The range ΔK is determined by computing and subtracting the maximum and minimum values of K in a given load cycle.

If ΔK is replaced by an effective stress intensity factor range,

$$\Delta K_{eff} = \frac{\Delta K(N, a(N))}{(1 - R(N, a(N)))^m}, \quad (3)$$

where the ratio R depends on the minimum and maximum values, K_{min} and K_{max} , respectively, of the stress intensity factor in a given load cycle,

$$R(N, a(N)) = \frac{K_{min}(N, a(N))}{K_{max}(N, a(N))}, \quad (4)$$

then the Walker equation [46,47] is obtained:

$$\frac{da}{dN} = C [\Delta K_{eff}(N, a(N))]^n, \text{ for } \Delta K(N, a(N)) > \Delta K_{th}. \quad (5)$$

The Walker and the Paris equations are two among the many possibilities used in practice to define da/dN . Some others may be found, for example, in [47,48].

In this paper, the crack propagation problem is seen in a more general way, writing the crack growth rates $\frac{da}{dN}$ in terms of a general crack propagation function, $f_{prop}(\cdot)$:

$$\frac{da}{dN} = f_{prop}(\Delta K(N, a(N)), R(N, a(N))). \quad (6)$$

For the sake of simplicity, the formulation is presented only as a function of the crack depth, a , although other dimensions of the crack could also be included. Also for simplicity, but without loss of generality, the threshold ΔK_{th} is assumed to be null during this explanation, such that crack growth occurs in any load cycle.

Starting from an initial time, t_0 , where the initial crack depth value, a_0 , is known. The crack depth at a given time, t_k , after the application of N_k load cycles is obtained by integrating the propagation function over the number of cycles or over time:

$$a(t_k) = a(N_k) = a_0 + \int_1^{N_k} f_{prop}(\Delta K(N, a(N)), R(N, a(N))) dN = a_0 + \int_{t_0}^{t_k} f_{prop}(\Delta K(N, a(N)), R(N, a(N))) \frac{dN}{dt} dt. \quad (7)$$

As indicated in [20], in general it is not possible to find analytical solutions for this integral, hence numerical integration schemes are necessary.

One way to perform the integration consists of taking a summation of N_k discrete terms, each one associated with one load cycle. In

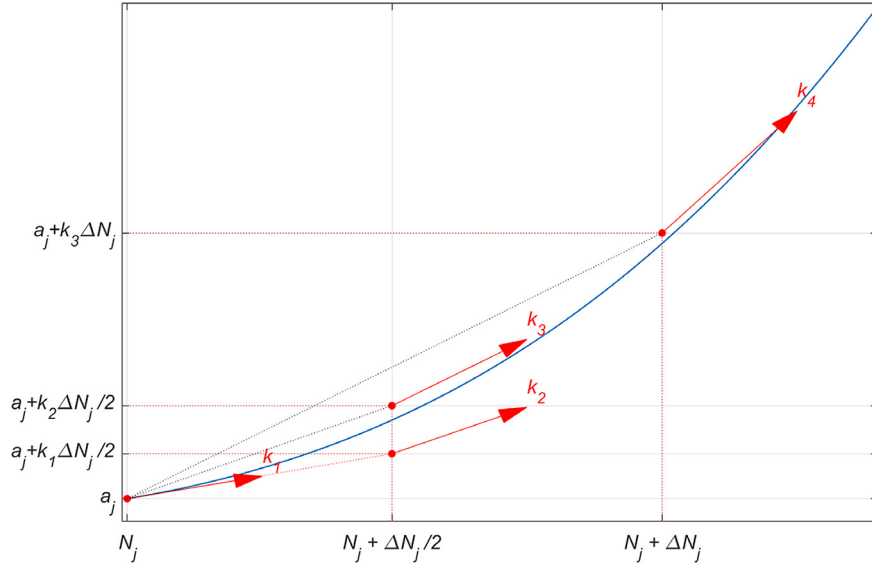


Fig. 1. Slopes used by the 4th order RK method (RK4).

this case, the numerical integration is done cycle by cycle and the procedure uses only the first order derivative. This leads to the well-known Euler method which is, in fact, an example of a general class of approximations known as Runge-Kutta methods [49]. The problem with the simple Euler method is that the derivative, in this case $f_{prop}(\cdot)$, is assumed constant over the entire step or cycle [49]. Although the step size may be decreased, in an attempt to increase the accuracy of the results, a very large number of steps may still accumulate errors and lead to convergence problems. According to [20], when dealing with crack propagation problems the first order Euler scheme can significantly underestimate the crack size.

In order to overcome this problem and achieve acceptable errors, as well as to allow employment of larger steps, higher order RK methods may be considered. In this case, in each iteration j the resulting crack depth, a_{j+1} , is determined by considering the variation of the crack depth, Δa_j , within the iteration:

$$a_{j+1} = a_j + \Delta a_j \quad (8)$$

A given number of load cycles, ΔN_j , is applied to the structural element during the iteration, so that the total number of cycles is updated by:

$$N_{j+1} = N_j + \Delta N_j \quad (9)$$

Different methods provide different estimates for Δa_j .

The fourth order RK method (RK4), also known as classical RK method, is the most famous, and the most widely used [14]; probably because it is usually accurate enough, easy to be implemented on computers and presents a relatively small computational cost. Via RK4, the integration is carried out using:

$$\Delta a_j^{RK4} = \Delta N_j \left(\frac{k_1}{6} + \frac{k_2}{3} + \frac{k_3}{3} + \frac{k_4}{6} \right) \quad (10)$$

where Δa_j^{RK4} depends on a weighted average of the slopes k_i , with $i = 1, 2, 3, 4$. These slopes, illustrated in Fig. 1, are obtained at certain points along the integration interval, according to:

$$k_1 = \frac{da_1}{dN} = f_{prop}(\Delta K(N_j, a_j), R(N_j, a_j)) \quad (11)$$

$$k_2 = \frac{da_2}{dN} = f_{prop} \left(\Delta K \left(N_j + \frac{\Delta N_j}{2}, a_j + k_1 \frac{\Delta N_j}{2} \right), R \left(N_j + \frac{\Delta N_j}{2}, a_j + k_1 \frac{\Delta N_j}{2} \right) \right), \quad (12)$$

$$k_3 = \frac{da_3}{dN} = f_{prop} \left(\Delta K \left(N_j + \frac{\Delta N_j}{2}, a_j + k_2 \frac{\Delta N_j}{2} \right), R \left(N_j + \frac{\Delta N_j}{2}, a_j + k_2 \frac{\Delta N_j}{2} \right) \right), \quad (13)$$

$$k_4 = \frac{da_4}{dN} = f_{prop}(\Delta K(N_j + \Delta N_j, a_j + k_3 \Delta N_j), R(N_j + \Delta N_j, a_j + k_3 \Delta N_j)) \quad (14)$$

After computation of the slopes, the crack size variation is determined via Eq. (10) and used to obtain the new crack size by using Eq. (8), with $\Delta a_j = \Delta a_j^{RK4}$. The iterative procedure continues until the total number of cycles is reached.

Note that, if the propagation function changes as the crack propagates, the slopes must be computed taking this into account. For example, if a transition occurs from a corner crack to a through crack, the propagation function would change and this should be considered when calculating the slopes, but integration would still be performed using the previous equations. This is also valid in what concerns effects which would have an impact on how the crack propagates, such as crack-closure effects, but which are not directly addressed herein.

3. Adaptive Runge-Kutta method

Writing the range ΔK as a function of the stress range over the cycle, $\Delta \sigma$, and of the geometry factor, $Y(\cdot)$, it is emphasized that ΔK in fact changes as the crack grows:

$$\Delta K(N, a) = Y(a) \Delta \sigma(N) \sqrt{\pi a} \quad (15)$$

Besides, note that $\Delta \sigma$, as well as R in the case ΔK_{eff} is employed, may change over the load cycles, so that they should be seen, in a general way, as functions of N . This implies that, during the computation of the slopes k_i , the values of $\Delta \sigma$ and R at the respective points must be considered. These points are defined by: N_j , for k_1 ; $N_j + \frac{\Delta N_j}{2}$, for k_2 and k_3 ; and $N_j + \Delta N_j$ for k_4 .

In order to obtain sufficiently accurate responses, it may be necessary to reduce the step size. However, this may introduce difficulties, even if higher order methods are employed.

Along the lifespan of the structural element, the load cycles may change significantly. This happens, for example, in risers used in the offshore oil and gas industry, which are elements responsible for carrying oil and gas extracted from subsea wells, up to the floating units on

the sea surface. These elements are subject to significant dynamic action and cyclic loads, over long service lives. As a consequence, stresses along the riser present an oscillatory behavior and significant variations may occur over time and space. If $\Delta\sigma$ significantly changes from a cycle to another, the crack growth rate may change abruptly, leading to errors in the integration. Other details related to the simulation, such as use of piecewise linear interpolations for the stress intensity factors, may also increase the possibility of having considerable changes, and even discontinuities, in the behavior of the crack growth curve.

Discontinuities in the differential equations of an initial value problem, and their effects on the results obtained by RK methods, have been discussed in the literature for a long time [28,34,50]. The standard procedure to increase accuracy consists of changing the step-size in an attempt to detect the discontinuities. This leads to the so-called adaptive methods, which try to automatically identify regions where significant changes in the behavior lead to errors in the integration. In crack propagation problems, these regions are basically those where the load changes enough to significantly change the crack growth rate. The adaptive methods are discussed further in the next subsections. After that, an algorithm to simulate loads which can enforce changes and discontinuities in the behavior of the crack growth curve is presented, so that it can be applied in the numerical examples of Section 4.

3.1. Adaptive RK method

In general, adaptive methods start by using a large step size. If the estimated error in the current iteration is greater than a given tolerance, the step size is reduced and then the integration process proceeds. The step size may also be increased in case the error is far below the tolerance. The integration continues until the total number of cycles is reached. This allows for refinement of the solution only where necessary, that is, in the regions where significant errors in the integration start to appear.

To establish an adaptive RK method, it is common to consider a pair of RK methods of different orders. If the classical RK4 method is employed, for example, it is possible to obtain third order approximations, via RK3, at small additional computational expense. The third order RK approximation depends on some of the slopes used to compute the RK4. The variation of the crack depth is given by:

$$\Delta a_j^{RK3} = \Delta N_j \left(\frac{k_1}{6} + \frac{2k_2}{3} + \frac{k_3}{6} \right) \quad (16)$$

Comparing the third and fourth order approximations one can obtain an estimate of the absolute error at the current step (Eq. (17)). The difference between the two solutions is an asymptotically correct estimate of the local error of the lower order solution [34].

$$e_j = |\Delta a_j^{RK4} - \Delta a_j^{RK3}| \quad (17)$$

If the estimated error, e_j , is larger than a tolerance, tol_e , the step is rejected and the step size must be reduced. One alternative consists of taking half the previous step size. This is done, for example, within the *ode45* function in MATLAB® [51]. Also, a limit in terms of a minimum acceptable step size is applied, here adopted equal to 1, to ensure that the step is not too small. The new step size, ΔN_j^{new} , is given by:

$$\Delta N_j^{new} = \max \left(\frac{\Delta N_j}{2}, 1 \right) \quad (18)$$

If the step is accepted, it is possible to try to increase the size of the next step in order to accelerate the integration process. The literature indicates the following equation to adjust the step size [21,52]:

$$\Delta N_j^{new} = \Delta N_j \cdot b \left(\frac{tol_e}{e_j} \right)^d \quad (19)$$

where the constants b and d are determined empirically. These constants are taken in the present paper as $b = 0.8$ and $d=1/5$, which are the same values used in *ode45* MATLAB function. It also important

to impose limits to the step size increase or reduction so that it does not result too big or too small. In terms of a lower limit, the MATLAB function *ode45* takes the maximum between 10% of ΔN_j and the value provided by Eq. (19), and this is also done here. In terms of an upper limit, it is also imposed here that the step size does not lead to a number of cycles greater than the total number of cycles and that the step size is not greater than one tenth of the total number of cycles.

The final equation to update the step size after a step is accepted, for a problem with a total of N_k load cycles and including the upper and lower bounds, is:

$$\Delta N_j^{new} = \min \left(\max \left(0.1 \cdot \Delta N_j, \Delta N_j \cdot b \left(\frac{tol_e}{e_j} \right)^d \right), \min \left(\frac{N_k}{10}, N_k - N_j \right) \right) \quad (20)$$

Even by considering the possibility of step size adjustment, note that the discontinuities may have similar impact on both components of the RK pair employed. This makes it difficult to obtain a good discretization over the number of cycles. In other words, this may lead to errors in both estimates, those of third and fourth orders, in such a way that the error defined by Eq. (17) becomes insufficient or inefficient for rejecting steps. This may lead to significant errors, mainly at the end of the integration process.

Determination of locations and discontinuities in the derivatives of functions is a problem related to many other areas and has been studied, for example, in [53]. However, in the problems of interest of the present paper, the detection must be performed in a simple way, with reduced computational effort: in stochastic crack growth analyses, it is common to have many simulations where the crack does not grow significantly. Application of refined analyses to detect changes in the crack growth behavior may lead to prohibitive and unnecessary computational costs. Furthermore, it is desired to have a solution strategy capable of dealing with millions of simultaneous simulations, so that vectorization and/or parallelization may be used, for example, leading to a reduction in the required computational time.

In this sense, a modification of the usual adaptive method is proposed.

3.2. Proposed adaptive method

The proposed adaptive method employs a simple analysis of the slopes k_i to obtain a procedure for step size adjustment, which is more suitable to deal with discontinuities in the derivative of the crack growth curve. These slopes are available at no additional computational costs during application of the RK method.

The idea explored herein resembles the one presented in [54], where the differential equation is added to the original differential system so that discontinuities may be detected, although there are differences in the way the slopes are taken into account here and in how the step size is controlled. From another point of view, the proposed method could also be compared with the method presented by Cash and Karp [55], which in each integration step uses a family of Runge–Kutta methods, with the possibility of accepting a fraction of the step size with a lower order RK or the total step size with a higher order RK. In this comparison, the proposed method is certainly easier to implement and to vectorize/parallelize, but it is less efficient if the simulations are performed one by one. Furthermore, the variable order method presented in [55] tends to be less efficient if function evaluations are fast, as stated by the authors, due to the time required to check the errors at the intermediate steps. The problems of interest of the present paper are those for which functions evaluations are fast and the computational effort comes from the large number of simulations required.

The proposed method starts with the definition of a standard deviation related to the slopes k_i (see Fig. 1). First the mean, k_m , of the

slopes k_2 and k_3 is taken, since both slopes are computed for a total of $(N_j + \frac{\Delta N_j}{2})$ cycles:

$$k_m = \frac{k_2 + k_3}{2} \quad (21)$$

The terms $(k_m - k_1)$ and $(k_4 - k_m)$ may be seen as direct estimates of how much the slope k changes from N_j to $N_j + \Delta N_j/2$ and from $N_j + \Delta N_j/2$ to $N_j + \Delta N_j$, respectively. The mean of these terms is given by:

$$\mu_{\Delta k} = \frac{(k_m - k_1) + (k_4 - k_m)}{2} \quad (22)$$

and the standard deviation is:

$$std_k = \sqrt{((k_m - k_1) - \mu_{\Delta k})^2 + ((k_4 - k_m) - \mu_{\Delta k})^2} \quad (23)$$

Note that the higher the standard deviation, the higher is the variation of the slope along the integration step, and there is a greater probability of having discontinuities within the interval related to this step. Thus, this indicates a possible necessity of step size reduction.

The standard deviation, std_k , is multiplied by the current step size, ΔN_j , and the result is interpreted as another indicator of errors, to be added to the error established by Eq. (19). This leads to a new error estimator, $\hat{e}_j$, which by utilizing the standard deviation std_k tries to also consider the possibility of having a discontinuity in the interval:

$$\hat{e}_j = 0.5 \cdot |\Delta a_j^{RK4} - \Delta a_j^{RK3}| + 0.5 \cdot |\Delta N_j \cdot std_k| \quad (24)$$

Note that different weights could be employed for each term. However, for the sake of simplicity, the same weight, 0.5, is considered herein for both terms. Although further investigation is necessary to better choose these weights, and they are probably problem dependent, preliminary results show that the algorithm is not very sensitive to them. The results were essentially the same for weights in the interval [0.4, 0.6], keeping the sum of the weights equal to one.

If a system of crack growth equations needs to be solved simultaneously, to consider growth of more than one crack geometry parameter, two possibilities arise. First, if discontinuities related to each parameter may occur at different cycles, the error estimator given by Eq. (24) must be modified to accommodate terms related to each parameter. Second, if every time a discontinuity occur for one or more parameters, discontinuities also occur for a particular parameter, the error estimator could focus only on this particular parameter. This way, all discontinuities may be correctly identified. In practice, the second case is probably the most common, since changes in the load are the main source of discontinuities and this reflects on all parameters at the same time.

In the proposed adaptive method, $\hat{e}_j$ replaces e_j , and is used to decide if the step is accepted or rejected. Also, a new equation based on the slopes is proposed, with the purpose of adjusting the step size in those cases where it is rejected.

The intervals $[N_j, N_j + \Delta N_j/2]$ and $[N_j + \Delta N_j/2, N_j + \Delta N_j]$ are called first and second intervals, respectively, and weights are associated to these intervals, w_1 to the first and w_2 to the second:

$$w_1 = \left(1 - \frac{k_m}{k_1}\right)^2 \quad (25)$$

$$w_2 = \left(1 - \frac{k_4}{k_m}\right)^2 \quad (26)$$

Note that, for example, the higher the difference between k_m and k_1 , the higher the variation of the slope in the first interval, and Eq. (25) results in a greater weight w_1 . In this case, and in the absence of further information which could be used to perform a more precise analysis, the ideal step size would be close to $\Delta N_j/4$, which corresponds to the center of the first interval. On the other hand, the higher the difference between k_m and k_4 , the higher is the variation of the slope in the second

interval and the higher the respective weight, w_2 . In this case, in the absence of further information, the ideal step size would be close to $3\Delta N_j/4$, which corresponds to the center of the second interval.

By combining both weights and considering the previous discussion, the following equation is obtained:

$$\Delta N_j^{new} = \max \left(\left(w_1 \cdot \frac{\Delta N_j}{4} + w_2 \cdot \frac{3 \cdot \Delta N_j}{4} \right) / (w_1 + w_2), 1 \right) \quad (27)$$

where a limit in terms of a minimum acceptable step size is applied, here adopted equal to 1, to ensure that the step is not too small.

If a system of crack growth equations needs to be solved simultaneously, to consider growth of more than one crack geometry parameter, Eq. (27) may be changed to accommodate contributions of all parameters. Weights could be computed for each parameter as well as step sizes, and ΔN_j^{new} could be taken as: the mean of the computed step sizes, in an average sense; or the minimum among the computed step sizes, in a conservative manner.

Eq. (27) tries to obtain, in an approximated way and using only already available information, an estimate for the ideal step size for cases where the step size is rejected in the current integration step. In the proposed adaptive method, Eq. (27) is used instead of Eq. (18). For accepted steps, Eq. (20) is kept.

Fig. 2 illustrates an example of the proposed method: the average slope k_m at $N_j + \Delta N_j/2$; the interval where ΔN_j^{new} is searched for, which is represented by the shaded region; the value found for ΔN_j^{new} . In the illustrated case, a discontinuity exists in the derivative of the curve within the interval $[N_j + \Delta N_j/2, N_j + \Delta N_j]$. As the difference between k_4 and k_m is larger than the difference between k_m and k_1 , Eq. (27) provides a value for ΔN_j^{new} which is closer to the discontinuity than the value $\Delta N_j/2$ provided by Eq. (18) in the original method. Obviously, a more refined search for the discontinuity could be applied, for example employing continuous extensions; however, this would require more information and greater computational effort.

The proposed method is adaptive and makes use of RK3 and RK4; thus, it is called $RK34_{Adapt}$ from now on. With some minor modifications, the same method could be applied for different RK pairs. This would be done by: computing the mean and standard deviations of the slope changes; estimating the error via Eq. (24), replacing Δa_j^{RK4} and Δa_j^{RK3} by the corresponding terms given by the new RK pair; defining new weights based on how the slopes change over the integration interval and a new way to combine them to define ΔN_j^{new} , that is, changing Eqs. (25) to (27) accordingly. Application of the proposed method in combination with higher order RK pairs may be the subject of future studies.

The pseudo-code for the proposed method is as follows. The processing is done in batches of samples (simulations), to require less RAM memory, and in a vectorized way to perform as many simultaneous simulations as possible.

(a) Initialize (at $N = 0$):

1. Set the values for the deterministic parameters of the problem;
2. Generate values for the random variables and stochastic processes of the problem, following their respective distributions and correlations;
3. Set the initial step size, ΔN_0 , as one-tenth of the total number of cycles, for all simulations in the batch;
4. Set all simulations as active.

(b) While the number of active simulations is greater than zero:

1. Apply RK to perform the integration considering the current step size, ΔN_j :
 - 1.1. Compute the slopes (Eqs. (11) to (14));
 - 1.2. Compute Δa_j^{RK3} (Eq. (16)), Δa_j^{RK4} (Eq. (10));

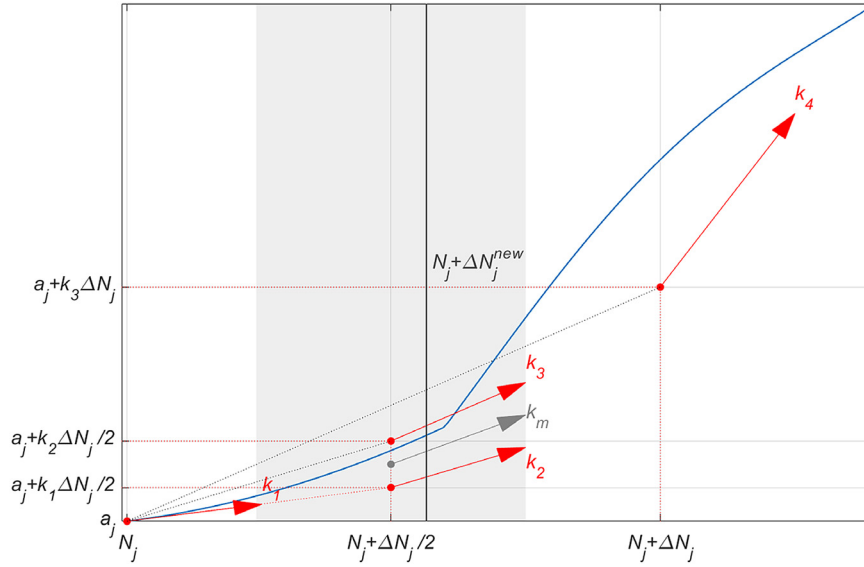


Fig. 2. Computation of ΔN_j^{new} considering its search interval (shaded region).

- 1.3. Compute the mean and standard deviation of the slope changes (Eqs. (22) and (23));
- 1.4. Compute the error (Eq. (24)).

2. Verify acceptance or rejection of the step:

- 2.1. For those samples for which the tolerance is not met:

- Compute the weights (Eqs. (25) and (26));
- Update the step size (Eq. (27)).

- 2.2. For those samples for which the tolerance is met:

- Update the crack size (Eq. (8)) and the number of processed cycles (Eq. (9));
- Update the step size (Eq. (20)).

3. Set as inactive the simulations for which the number of processed cycles is equal to the total number of cycles.

- (c) Return the crack sizes over the number of cycles, as well as the time-step discretization in terms of number of cycles related to each computed crack size.

3.3. Comparison between the traditional and the proposed adaptive RK methods

This section presents a brief and simple comparison between the adaptive RK method described in Section 3.1 and the adaptive method proposed in Section 3.2. These methods differ only in the way the step size is updated when rejected (see Eqs. (18) and (27)), and in the error estimator applied to decide if the step size is accepted or not (see Eqs. (17) and (24)). Due to the similarities between them, the proposed method may be applied to the same kind of problems that the traditional method is able to solve.

The effect of these differences is illustrated by taking a simple example, with $a_0 = 1$ mm, where the Paris' law is employed with fixed parameters $C = 10^{-10}$ and $n = 3$, and the geometry factor is assumed to be constant and equal to 1. Four load cycles are considered, with $\Delta\sigma$ changing from 600 MPa, during the first three cycles, to 1000 MPa during the last cycle. The tolerance on the error is taken as 10^{-6} , for both methods.

The initial step size is $\Delta N = 4$, in an attempt to directly compute the final results. As can be seen in Fig. 3, the discontinuity in the derivative

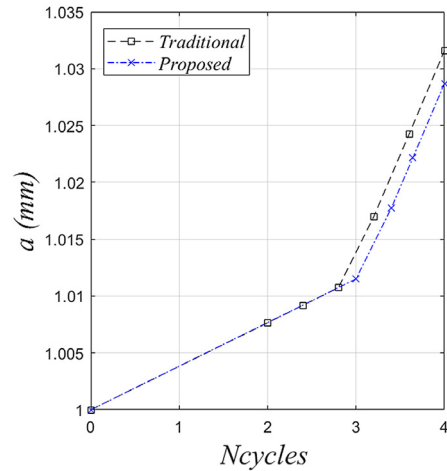


Fig. 3. Comparison between the traditional and the proposed adaptive methods.

of the crack growth curve occurs when the third cycle finishes and the last cycle initiates.

The traditional method tries to apply the initial step size, but the error results greater than the tolerance, so that the step size is updated to $\Delta N = 2$ using Eq. (18). The algorithm inserts a point at $N = 2$, and, with a successful step, tries to increase the step size to 4.27, but one of the upper limits included in Eq. (20) is activated, so that ΔN results equal to 0.4, which consists of one tenth of the total number of cycles. The next steps are applied using this step size and, each time the algorithm tries to increase the step size after a successful step, the same upper limit is activated.

The proposed method tries to apply the initial step size, but the step is rejected. The step size is updated via Eq. (27), which in this case leads to the inclusion of a point almost at $N = 3$, where the discontinuity appears. After that all steps are successful and ΔN is limited by one tenth of the total number of cycles.

It is seen that, although simple, the idea of trying to locate the discontinuity using the slopes when the step is rejected may lead to better results than just taking half of the previous step size. Also, if there are no discontinuities, ΔN_j^{new} provided by the proposed method

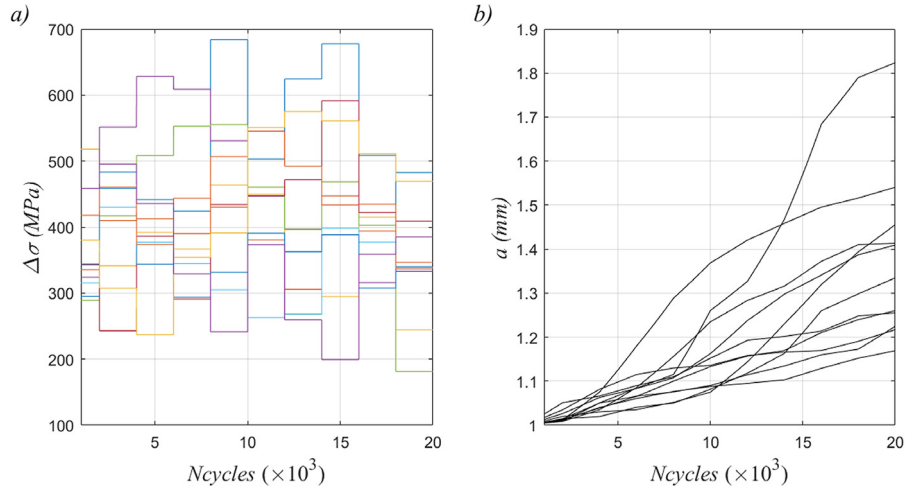


Fig. 4. (a) Load simulations and (b) crack growth curves.

would be very close to the one provided by the traditional method, so that similar final results would be achieved.

3.4. Other adaptive methods for comparison purposes

The literature provides many adaptive methods to solve IVPs. Many of them make use of RK pairs in order to estimate the error, and present procedures to try to define the ideal step size. It is the case of the methods implemented in the MATLAB functions *ode23s* and *ode45*, two of the most used functions among those available in the ordinary differential equation MATLAB toolbox. After testing many functions of this toolbox in solution of crack propagation problems, the functions with most accurate results and greater efficiency were found to be *ode23s* and *ode45*. These functions are employed herein to compare results with the proposed method.

MATLAB presents an implementation of the RK pair of second and third order, given in the *ode23* function. This pair is known as the Bogacki and Shampine pair [56]. On the other hand, the *ode23s* function focuses on stiff differential equations, which are those for which numerical solutions are unstable, unless a sufficiently small step size is taken. A discussion about stiffness of differential equations may be found in [57]. The method implemented in the *ode23s* function is based on a modified Rosenbrock formula of order 2, and consists on a variation of the implicit RK method [51]. Being an implicit method facilitates convergence and helps identifying discontinuities, even if relatively large step sizes are taken. However, it becomes impossible or impractical to make use of vectorization, once each simulation requires, at each step, the solution of a system of equations.

It would be possible to reuse the matrix inverses which appear in the solution of the system if the same discretization was used for all simulations. However, in stochastic crack growth analyses, it is common to have very different simulations, some of them leading to large cracks and many of them not. Thus, use of the same discretization implies an excessive number of steps for the simulations where the crack slightly grows, and/or an insufficient number of steps for the simulations where the crack significantly grows.

Besides, it is not known *a priori* which simulations will lead or not to significant propagation. It becomes difficult to divide the simulations by similar behavior. Thus, the *ode23s* function needs to be applied simulation by simulation, reducing its overall efficiency.

In contrast to *ode23s*, the proposed adaptive method is explicit, since the equations to compute Δa_j depend only on a_j , and not on a_{j+1} . As a consequence, the method becomes easier to implement, but prone to have stability problems depending on the differential equation to be solved. As the method present in the *ode23s* function is implicit

and stable, it is possible to verify both stability and accuracy of the *RK34_{Adapt}* by comparing its results with those obtained via *ode23s*.

The *ode45* function, on the other hand, uses the Dormand and Prince pair [58], with RKs of orders 4 and 5. It is an explicit adaptive method, very similar to what was presented in Section 3.1. However, the way the method was implemented in *ode45* does not allow for vectorization, unless the same discretization is used, which is not desirable as previously discussed. Therefore, a new implementation was made based on the *ode45* function so that simultaneous simulations with different discretizations would be possible. Also, to have more coherent comparisons, the classical RK4 method and its pair, RK3, are employed, instead of the pair of orders 4 and 5. The implemented function is called *ode34* herein, and is very similar to the *ode45* MATLAB function.

3.5. Stochastic load simulation

In order to verify if the adaptive methods are capable of dealing with discontinuities and significant changes in the derivatives of the crack propagation curve, a discrete load whose intensity changes every $N_{c\Delta\sigma}$ cycles is adopted. The load is characterized by the stress range over the cycle, $\Delta\sigma$, which is taken at first as normally distributed, with mean $\mu_{\Delta\sigma}$ and coefficient of variation $cov_{\Delta\sigma}$. An exponential correlation along the blocks of $N_{c\Delta\sigma}$ cycles is also adopted, with a correlation function given by:

$$f = e^{-1/\tau}, \quad (28)$$

where τ is a parameter defining the correlation length.

A recursive algorithm to generate a sequence of exponentially correlated numbers, used to define $\Delta\sigma$ in each $(i+1)$ block of cycles, is obtained from [59] and given by:

$$\Delta\sigma_{i+1} = f \cdot \Delta\sigma_i + \sqrt{1 - f^2} \cdot \Delta\sigma_{aux}, \quad (29)$$

where $\Delta\sigma_{aux}$ is randomly generated following the distribution of $\Delta\sigma$, and $\Delta\sigma_0$ is taken as null to generate the value for the first block, $\Delta\sigma_1$.

Fig. 4 illustrates 10 simulated load scenarios and the impact they may have on the crack propagation curves, leading to abrupt changes in the derivatives. In this case, the parameters were $N_{c\Delta\sigma} = 2000$ cycles, $\tau = 1$, $\mu_{\Delta\sigma} = 400$ MPa and $cov_{\Delta\sigma} = 0.25$.

Note that, in practice, $\Delta\sigma$ may change cycle by cycle. However, for efficient stochastic crack growth computation, the algorithm must be able to reduce the step size only if these changes have significant impact on the crack propagation. This depends not only on the load, but also on the geometry function, on the current crack dimensions, and other variables. Thus, the decision on where to insert points in the mesh cannot be taken by looking at the load history only.

Table 1
Parameters for the first example.

Parameter	Description		Distribution type	Mean/Value	Cov
$\Delta\sigma$ (MPa)	Stress range over the cycle	Cases I and II	Normal	350	0.25
		Cases III and IV	Normal	400	0.25
r	Correlation length parameter	Cases I and III	–	10	–
		Cases II and IV	–	20	–
a_0 (m)	Initial crack depth		–	0.001	–
n	Paris' Law parameter		–	3.0	–
C	Paris' Law parameter		–	10^{-12}	–
ΔK_{th} (MPa $\sqrt{m}$)	Fatigue threshold		–	0	–
N_i	Total number of cycles		–	$20 \cdot 10^3$	–
$N_{c,\Delta\sigma}$	The load changes every $N_{c,\Delta\sigma}$ cycles		–	$4 \cdot 10^3$	–

4. Numerical examples

Three examples are presented here, to illustrate the applicability of the proposed adaptive method to the stochastic crack propagation problem. The first and second examples consist of plates loaded in tension for which analytical geometry functions are available: a center-cracked infinite plate and an edge-cracked finite plate. Different cases are analyzed, varying some parameters of the problems, to investigate the applicability of the methods under different situations. For both examples, Paris' law is employed with a null threshold ΔK_{th} . The third example is related to weld flaws in pipes, and aims at a problem which is closer to real-world applications, where lots of load cycles are applied and the stress intensity factors are computed based on previous finite element analyses.

Solutions obtained cycle by cycle via $RK34$ are also computed herein and called $RK34_{Fixed}$ since they use a fixed step size. They are computationally expensive but also tend to be very accurate, and may be taken as reference solutions for problems where no analytical solutions are available.

The tolerances related to each method were adjusted so that all methods would reach similar accuracy in the final results. For $ode23s$ the tolerances are 10^{-6} and $8 \cdot 10^{-9}$ for the relative and absolute errors, respectively. For $ode34$, the tolerance on the error given by Eq. (17) is $4 \cdot 10^{-10}$. For $RK34_{Adapt}$, the tolerance on the error given by Eq. (24) is 10^{-7} . $RK34_{Fixed}$ does not employ errors nor tolerances.

The computational codes were implemented in MATLAB® and the computations were performed on an Intel® Core™ i7 CPU 860@2.80 GHz processor with 16 GB of RAM. The methods employed herein may be easily parallelized, by performing a number of Monte Carlo simulations in each CPU core. However, single-core computation is employed here to focus on the comparison of the vectorization capabilities. For the vectorized algorithms $ode34$ and $RK34_{Adapt}$, which require more memory, processing is done in batches of 10 simulations. $RK34_{Fixed}$ also works in a vectorized way, but requires less memory, so it processes all simulations at once, although cycle by cycle. And $ode23s$ is not vectorized as previously discussed.

4.1. Example 1: center-cracked infinite plate

This problem consists of a through crack in a plate of infinite width subjected to a remote tensile stress, as illustrated in Fig. 5. The problem is based on the example presented in [20], but has also been studied in other references from the literature (for example, [39]).

It is one of a few cases where the stress intensity factor is given in closed-form [39]:

$$K = \sigma \sqrt{\pi a} \quad (30)$$

By comparing Eq. (30) with Eq. (15), it is seen that the geometry function is constant and equal to one for this case. If Paris' law is considered, for example, solution follows example 10.1 of [39]. The

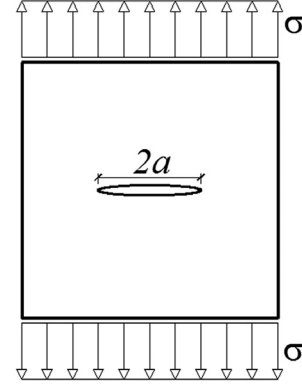


Fig. 5. Through crack in an infinite plate subjected to a remote tensile stress.

solution for this problem may also be found in [20], and needs to be applied cycle by cycle if the load changes cycle by cycle.

Another way to analytically determine the crack size in this case consists of first calculating an Equivalent Constant Amplitude loading and then applying it during the integration process, instead of the actual load. This procedure is presented, for example, in [21], and is also adopted herein. By using Equivalent Constant Amplitude, only the final response may be taken as reference, because the original load is replaced by one which has the same effect over the total number of cycles. The responses may differ along the cycles.

The parameters for this example are summarized in Table 1, with most of them taken as deterministic. Two different values are adopted for the load mean and two different correlations are considered for the load blocks over time. Higher values of r correspond to stronger correlations. The parameters for the Paris' law are obtained from [20].

Table 2 presents the results for all 4 cases and a total of 1000 simulations. It is seen that the proposed method requires a significantly lower computational time than the other methods. Also, $RK34_{Adapt}$ requires a similar number of calls to the function being integrated as $ode23s$, although the latter depends on the solution of systems of equations, demanding a higher computational effort. Note that when $RK34_{Fixed}$ is employed the Paris' equation is called 4 times per cycle, with a total of $80 \cdot 10^3$ calls, N_{calls} , per crack growth curve simulation.

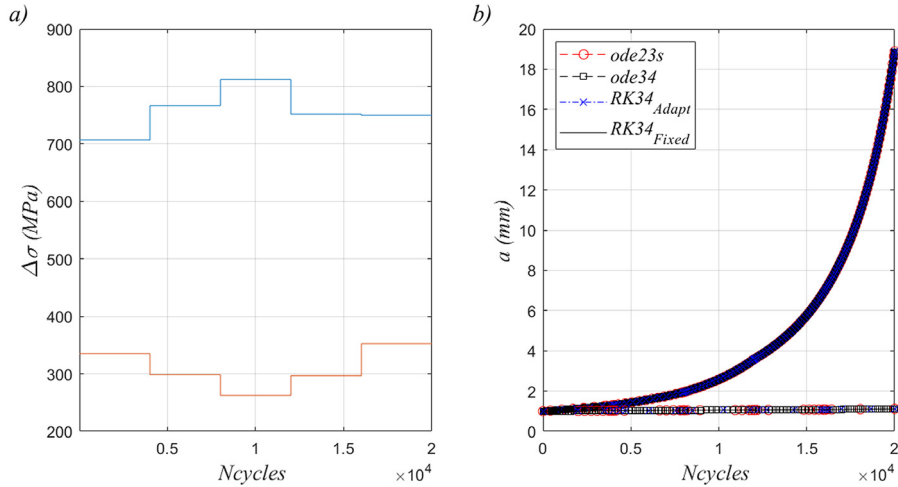
Fig. 6 illustrates, for case IV, the worst simulated scenario, which is the one with larger final crack depth, as well as another scenario randomly selected among all the simulations. For the randomly selected scenario, $\Delta\sigma$ does not reach sufficiently high values to trigger the crack propagation, so that a remains close to zero. In terms of the final crack size, very similar results are obtained by all methods in both scenarios.

To investigate the discretizations obtained, tolerances for all methods were changed so that average errors between 0.1% and 0.2% would be achieved for the final value a_f . This leads to tolerances of: 10^{-6} and $1.6 \cdot 10^{-6}$ for the relative and absolute errors of $ode23s$, respectively; 4

Table 2

Results for the first example.

Case	$\mu_{\Delta\sigma}$	τ	Method	Maximum a_f (mm)	Averages over N_{MC} simulations					Total time (s)
					Error (%)		NSteps ^a		NCalls ^b	
					μ	σ	μ	σ		
I	350	10	Analytical	4.2529	–	–	–	–	–	–
			ode23s	4.2535	0.0020	0.00	40.8	7.4	218.68	11.26
			ode34	4.2529	0.0061	0.01	211.9	173.3	861.60	7.01
			$RK34_{Adapt}$	4.2528	0.0021	0.00	57.4	13.3	231.49	1.67
			$RK34_{Fixed}$	4.2529	0.0001	0.00	1001.0	0.0	80000	16.53
II	350	20	Analytical	4.2638	–	–	–	–	–	–
			ode23s	4.2645	0.0021	0.00	38.1	7.4	204.71	10.50
			ode34	4.2639	0.0047	0.00	210.9	181.3	857.56	6.78
			$RK34_{Adapt}$	4.2638	0.0021	0.00	53.5	13.9	215.87	1.57
			$RK34_{Fixed}$	4.2638	0.0001	0.00	1001.0	0.0	80000	16.71
III	400	10	Analytical	18.7216	–	–	–	–	–	–
			ode23s	18.7269	0.0027	0.00	45.5	9.9	240.49	12.59
			ode34	18.7173	0.0064	0.01	346.4	381.2	1401.76	12.93
			$RK34_{Adapt}$	18.7207	0.0021	0.00	68.2	25.0	275.81	2.11
			$RK34_{Fixed}$	18.7216	0.0001	0.00	1001.0	0.0	80000	16.61
IV	400	20	Analytical	18.8730	–	–	–	–	–	–
			ode23s	18.8795	0.0027	0.00	42.3	10.0	224.66	11.53
			ode34	18.8717	0.0048	0.00	347.7	407.9	1406.87	12.22
			$RK34_{Adapt}$	18.8720	0.0020	0.00	64.4	26.7	260.54	2.25
			$RK34_{Fixed}$	18.8732	0.0001	0.00	1001.0	0.0	80000	16.85

^aNumber of steps required by the method.^bNumber of calls to the $f_{prop}(\cdot)$ function.**Fig. 6.** Simulations for case IV, including the worst scenario and another randomly selected one. (a) Simulated load over the number of cycles; (b) crack size along the number of cycles.

$\cdot 10^{-6}$ for the error of *ode34*; and $2 \cdot 10^{-5}$ for the error of *RK34_{Adapt}*. Fig. 7 illustrates this for the worst simulated scenario. It is seen how the methods, especially *ode23s* and *RK34_{Adapt}*, are able to refine the discretization near the regions where the load significantly changes and impacts the crack propagation curve. This occurs near $8 \cdot 10^3$ and $12 \cdot 10^3$ load cycles in this case, for this simulation.

Finally, it is analyzed how the computational time changes with the total number of simulations. This is done for the Case IV, although similar curves are obtained for all other cases. Fig. 8 shows how the proposed method becomes relatively more efficient than the other methods when the number of samples is increased. This is particularly desirable in the context of stochastic crack growth analysis.

4.2. Example 2: single-edge-cracked plate under uniform tension

This example consists of a single-edge-cracked plate under uniform tension (Fig. 9) and is similar to the second example presented in [36].

The geometry function used to compute the stress intensity factor for this case is given in the literature [60,61] as a fourth-order polynomial:

$$Y(a, N) = 1.122 - 0.231 \left(\frac{a(N)}{b} \right) + 10.55 \left(\frac{a(N)}{b} \right)^2 - 21.72 \left(\frac{a(N)}{b} \right)^3 + 30.39 \left(\frac{a(N)}{b} \right)^4. \quad (31)$$

There is no closed-form solution for the crack size, a , after a given number of load cycles takes place, in such a way that the RK34 cycle by cycle solution is taken here as a reference solution.

Table 3 summarizes the parameters for this example, with four different load cases.

Table 4 presents the results of all methods for all cases and a total of 1000 simulations. In all cases the proposed method required lower computational times and led to results similar to the reference,

Table 3
Parameters for the second example.

Parameter	Description		Distribution type	Mean/Value	Cov
$\Delta\sigma$ (MPa)	Stress range over the cycle	Cases I and II	Normal	350	0.3
		Cases III and IV	Normal	400	0.3
τ	Correlation length parameter	Cases I and III	–	10	–
		Cases II and IV	–	20	–
a_0 (m)	Initial crack depth		Lognormal	$1.5 \cdot 10^{-4}$	0.5
b (m)	Plate width		–	0.05	–
n	Paris' Law parameter		–	2.0	–
C	Paris' Law parameter		Lognormal	510^{-12}	0.8
ΔK_{th} (MPa $\sqrt{m}$)	Fatigue threshold		–	0	–
N_k	Total number of cycles		–	$90 \cdot 10^6$	–
$N_{c\Delta\sigma}$	The load changes every $N_{c\Delta\sigma}$ cycles		–	$9 \cdot 10^3$	–

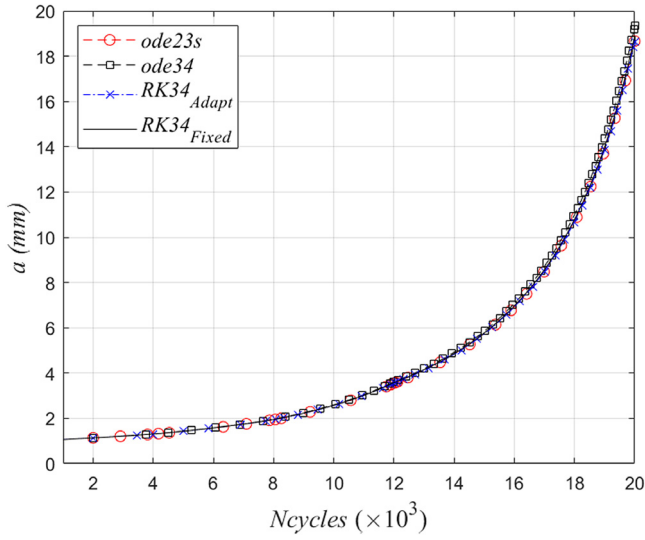


Fig. 7. Simulations for Case IV. The worst simulated scenario, considering higher tolerances for all methods.

with averages and standard deviations of the error very close to those obtained by the other methods.

Fig. 10 illustrates two simulations, including the worst simulated scenario among all simulations as well as a randomly selected simulation, for the third load case. Again, it is seen how close the results of the different methods are. If the load correlation is increased, the results remain very close, as illustrated in Fig. 11 for case IV.

Finally, an investigation of the computational times as the number of simulations increases (Fig. 12) shows that the proposed method is even more advantageous in comparison with the other methods, if a large number of simulations is required.

4.3. Example 3: weld flaws in a pipe

This problem is based on the second example of [62] and is related to weld flaws in pipes. It aims at addressing a situation closer to what is often found in practice, where stress intensity factors are computed using finite element models in a direct manner or by interpolation of previously computed results, and the pipe is subject to a high number of load cycles.

Semi-elliptical circumferential surface cracks are considered, as illustrated in Fig. 13, and the pipe is under static pressure, membrane and bending stresses, which are combined to determine the stress ranges over the cycles, $\Delta\sigma$, as indicated in API579-1 [47]. Crack growth in depth, a , and length, $2c$, are considered by applying a bilinear Paris' law, which employs different material constants, C and n , for different

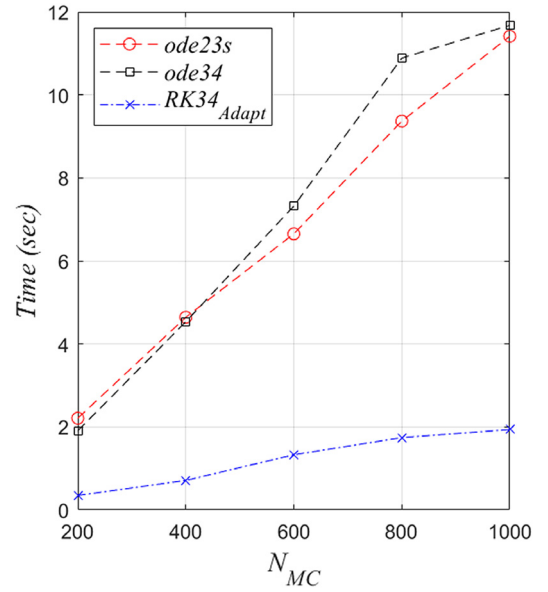


Fig. 8. Computational time as a function of the total number of simulations.

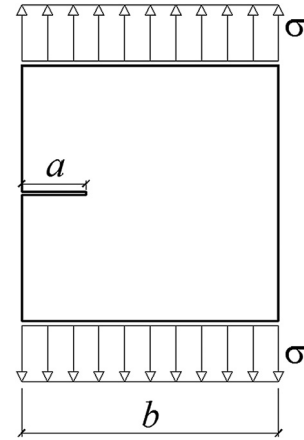


Fig. 9. Single-edge-cracked plate under uniform tension.

levels of ΔK , depending on two different fatigue thresholds, ΔK_{th1} and ΔK_{th2} . This is an attempt to obtain a better representation of different regimes of fatigue crack growth.

The proposed method is applied for crack growth computations in depth and length, but focusing at a , which has a greater impact on the safety of such structures. That is, Eq. (24) is used to compute errors

Table 4
Results for the second example.

Case	$\mu_{\Delta\sigma}$	τ	Method	Maximum a_f (mm)	Averages over N_{MC} simulations					Total time (s)
					Error (%)		NSteps ^a		NCalls ^b	
					μ	σ	μ	σ		
I	350	10	ode23s	3.4634	0.0212	0.02	42.1	15.8	240.83	13.39
			ode34	3.4628	0.0255	0.02	104.0	133.2	425.24	6.66
			RK34 ^{Adapt}	3.4629	0.0231	0.02	53.2	23.7	219.76	3.84
			RK34 ^{Fixed}	3.4629	–	–	1001.0	0.0	360000	27.05
II	350	20	ode23s	4.4125	0.0199	0.02	37.6	15.1	214.73	11.97
			ode34	4.4115	0.0191	0.02	103.3	140.1	422.53	6.72
			RK34 ^{Adapt}	4.4119	0.0229	0.02	46.1	23.0	190.97	3.84
			RK34 ^{Fixed}	4.4118	–	–	1001.0	0.0	360000	27.41
III	400	10	ode23s	6.3825	0.0209	0.02	46.6	17.4	265.27	14.16
			ode34	6.3820	0.0259	0.02	138.3	208.5	563.74	7.72
			RK34 ^{Adapt}	6.3815	0.0228	0.02	60.5	27.3	249.51	3.82
			RK34 ^{Fixed}	6.3813	–	–	1001.0	0.0	360000	25.69
IV	400	20	ode23s	9.3007	0.0207	0.02	41.5	16.2	236.96	12.91
			ode34	9.2985	0.0200	0.02	138.5	224.0	564.70	8.37
			RK34 ^{Adapt}	9.2988	0.0210	0.02	52.9	27.1	219.02	3.73
			RK34 ^{Fixed}	9.2988	–	–	1001.0	0.0	360000	26.76

^aNumber of steps required by the method.

^bNumber of calls to the $f_{prop}(\cdot)$ function.

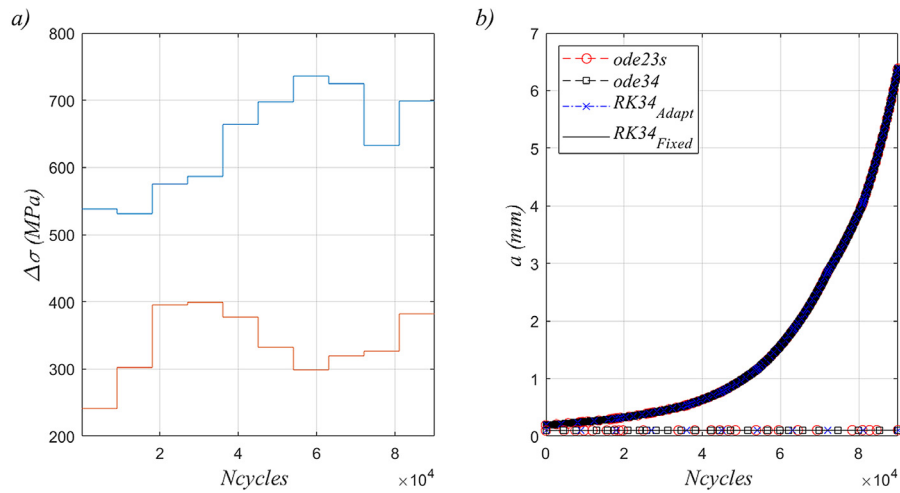


Fig. 10. Simulations for case III, including the worst scenario and a randomly selected one. (a) Simulated load over the number of cycles; (b) crack size along the number of cycles.

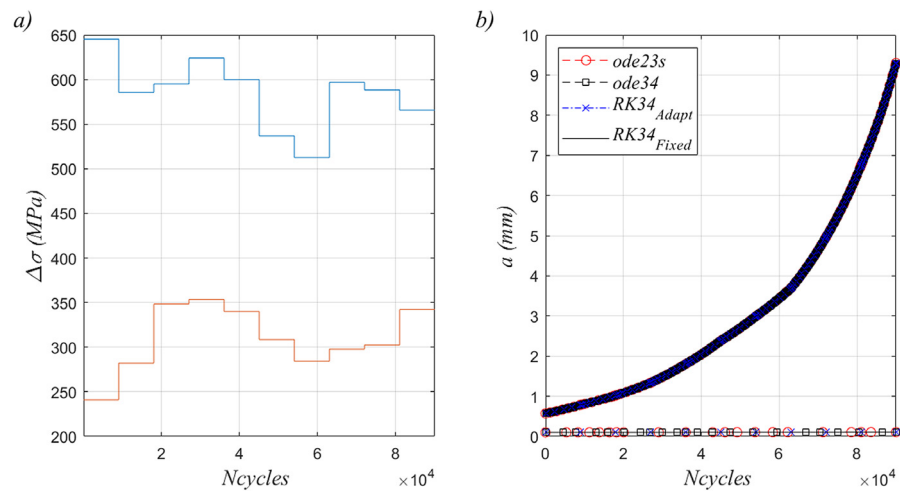


Fig. 11. Simulations for case IV, including the worst scenario and a randomly selected one. (a) Simulated load over the number of cycles; (b) crack size along the number of cycles.

Table 5
Parameters for the third example.

Parameter	Description	Distribution type	Mean/Value	Cov
ΔP_c (MPa)	Static pressure (variation)	Normal	16	0.1
$\Delta \sigma_b$ (MPa)	Bending stress (variation)	Normal	328	0.1
$\Delta \sigma_m$ (MPa)	Membrane stress (variation)	Normal	48	0.1
τ	Correlation length parameter	–	0	–
a_0 (m)	Initial crack depth	Lognormal	0.00254	0.25
a_0/c	Initial aspect ratio. $2c$ is the crack length	Lognormal	0.15	0.1
t (m)	Thickness of the pipe	Normal	0.0254	0.01
d_o (m)	External diameter of the pipe	–	0.508	–
n_1	Paris' Law parameter	–	5.1	–
n_2	Paris' Law parameter	–	3.0	–
C_1	Paris' Law parameter	Lognormal	2.1×10^{-17}	0.15
C_2	Paris' Law parameter	Lognormal	2.8×10^{-13}	0.15
ΔK_{th1} (MPa $\sqrt{m}$)	First threshold stress-intensity range	–	17.29	–
ΔK_{th2} (MPa $\sqrt{m}$)	Second threshold stress-intensity range (transition threshold)	–	28	–
J_{mat} (kJ/m ²)	Fracture toughness	Lognormal	30	0.1
f_y (MPa)	Yielding stress	Lognormal	470	0.1
E (GPa)	Elastic modulus	–	207	–
ν	Poisson's ratio	–	0.3	–
N_k	Total number of cycles	–	$2.5 \cdot 10^5$	–
$N_{c\Delta\sigma}$	The load changes every $N_{c\Delta\sigma}$ cycles	–	$5 \cdot 10^3$	–

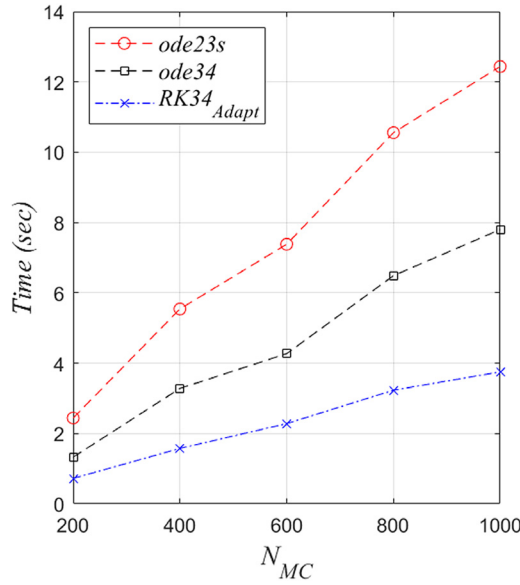


Fig. 12. Computational time as a function of the total number of simulations.

related to a , and to decide if the step should be accepted or rejected. Note that terms related to c , similar to the ones related to a , could also be included in Eq. (24), but preliminary results showed that this was not necessary since the discontinuities have impact on both a and c at the same load cycles. A similar decision is taken when applying *ode34*, where the error is estimated via Eq. (17) during the adaptive process, although a term related to c could also be included in this equation. For *RK34_Fixed*, solutions are obtained for a and c , cycle by cycle, without the need to estimate errors. Finally, *ode23s* allows for consideration of systems of differential equations, therefore dealing with both a and c simultaneously, as the matrices involved in the solution of the system of equations are constructed and inverted. d_0

A description of the random variables and deterministic parameters for this problem is presented in Table 5. Parameters C_1 and C_2 , of the bilinear Paris' Law, are assumed to be perfectly correlated, as well as the initial crack depth and length, as was done in [62]. An aspect ratio

a_0/c_0 is considered, so that the initial crack length c_0 is computed from a_0 during the simulations.

In comparison with the first case presented in [62], here the number of cycles is reduced from 2.5×10^6 to 2.5×10^5 , so that a reference cycle by cycle solution may be obtained by *RK34_Fixed* without an excessive computational cost. Also, the mean values of the loads are multiplied by a factor of 1.6 to lead to the values presented in Table 5, so that the cracks grow enough despite of the reduction on the number of load cycles. For each simulation, i , the range of the stress intensity factor per cycle, ΔK_i , is given as a function of the static pressure, the bending and membrane stresses, and is kept constant in each interval of $N_{c\Delta\sigma}$ cycles (see [62] for details). Loads are generated disregarding correlation, using the procedure described in Section 3.4 with $\tau = 0$, and following the distributions given in Table 5.

As seen in Table 6, results obtained by all methods, for the final crack depth and length, a_f and c_f , are very close. However, the proposed method requires less calls to the $f_{prop}(\cdot)$ function, hence smaller computation time. Fig. 14 illustrates the loads and crack size over time, for the simulation which achieved the maximum value of a_f , as well as for another randomly selected simulation. In this example, the discontinuities are not well pronounced, even though the proposed method is still as precise as the others and less time consuming.

Finally, Fig. 15 shows how the proposed method becomes more advantageous than the others as the number of required simulations grows. However, as the discontinuities in the crack growth derivatives are not as significant as in the previous examples, smaller differences are found between *ode34* and *RK34_Adapt* in terms of computational time.

5. Conclusions

In this paper a modified adaptive Runge–Kutta approach was proposed focusing on the efficient solution of stochastic crack growth problems. The approach was developed aiming at the simultaneous computation of thousands or millions of crack growth sample realizations, and at an inexpensive and vectorized procedure for the identification of the ideal step size in cases where the crack growth curve is continuous but not differentiable in one or more points. The procedure was based on the slopes already computed during the evaluation of the third and fourth orders RK pair.

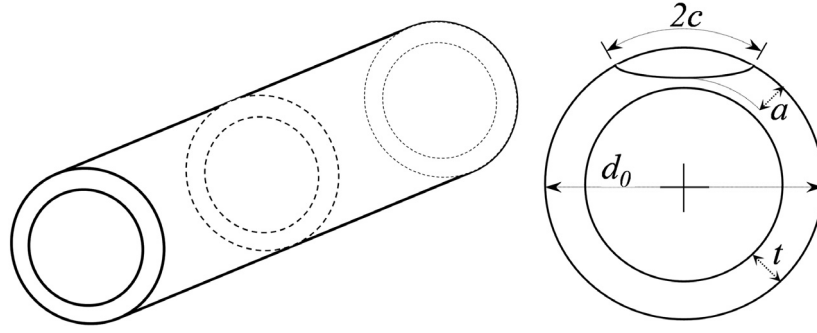


Fig. 13. Illustration of the circumferential surface crack in the pipe.

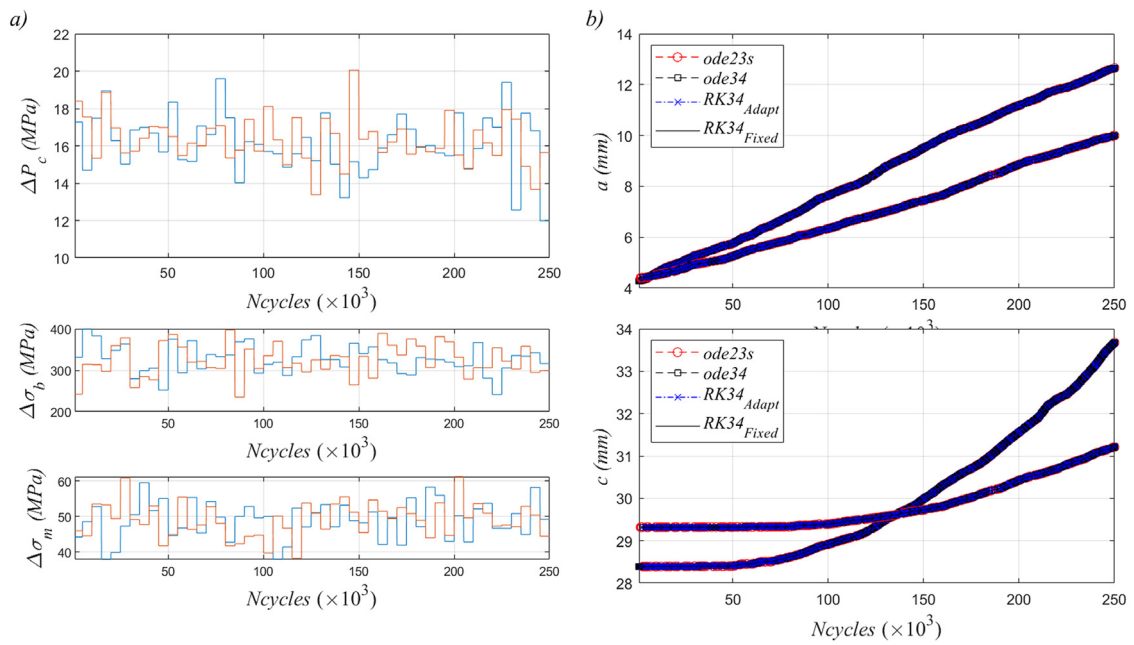


Fig. 14. Simulations for the worst and for a randomly selected scenario, third example. (a) Simulated loads over the number of cycles; (b) crack depth and length along the number of cycles.

Table 6
Results for the third example.

Method	Maximum a_f (mm)	Maximum c_f (mm)	Averages over N_{MC} simulations								Total time (s)		
			a_f				c_f					NSteps ^a	NCalls ^b
			Error (%)		Error (%)								
			μ	σ	μ	σ	μ	σ					
<i>ode23s</i>	12.6577	35.9849	0.0014	0.00	0.0004	0.00	586.0	42.3	3755.60	649.51			
<i>ode34</i>	12.6593	35.9838	0.2301	0.54	0.0397	0.07	1066.5	200.1	4296.72	97.91			
<i>RK34_{Adapt}</i>	12.6578	35.9828	0.0886	0.12	0.0206	0.03	851.4	48.2	3462.86	70.59			
<i>RK34_{Fixed}</i>	12.6577	35.9850	–	–	–	–	1001.0	0.0	1000000	812.18			

^aNumber of steps required by the method.^bNumber of calls to the $f_{prop}(\cdot)$ function.

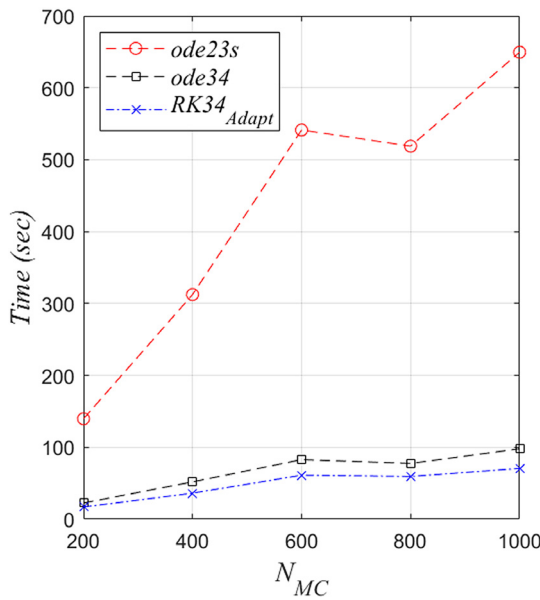


Fig. 15. Computational time as a function of the total number of simulations.

Application of the proposed method to three crack propagation examples indicated that it is as accurate as other adaptive methods from the literature, while requiring only a fraction of the computational time. The advantages of the proposed method become even more apparent when thousands of simulations are required, which is usually the case in stochastic crack growth analysis.

CRediT authorship contribution statement

Wellison José de Santana Gomes: Conceptualization, Methodology, Software, Writing – original draft, Writing – review & editing. **André Teófilo Beck:** Conceptualization, Methodology, Writing – review & editing. **Alexandre Galiani Garmbis:** Conceptualization, Writing – review & editing.

Declaration of competing interest

The authors have no conflict of interest in submitting this manuscript.

Data availability

Data will be made available on request.

Acknowledgments

This work was sponsored by PETROBRAS, Brazil through Cooperation Term 5900.0112628.19.9: “Innovation for acceptability of flaws in rigid risers using probabilistic fracture mechanics”. This work is also supported by the National Council for Technological and Scientific Development (CNPq), Brazil via grants 301756/2022-8 and 309107/2020-2.

References

- [1] D.F. Griffiths, D.J. Higham, Numerical Methods for Ordinary Differential Equations: Initial Value Problems, Springer, 2010, <http://dx.doi.org/10.1007/978-0-85729-148-6>.
- [2] J. Baker, R. Puzio, New method for solving the initial value problem with application to multiple black holes, Phys. Rev. D 59 (1999) 044030, <http://dx.doi.org/10.1103/PhysRevD.59.044030>.
- [3] A.S. Khoojine, M. Mahsuli, M. Shadabfar, V.R. Hosseini, H. Kordestani, A proposed fractional dynamic system and Monte Carlo-based back analysis for simulating the spreading profile of COVID-19, Eur. Phys. J. Spec. Top. (2022) <http://dx.doi.org/10.1140/epjs/s11734-022-00538-1>.
- [4] M.A. Mukaddam, M.B. Kasti, Reinforced concrete joints under cyclic loading, J. Struct. Eng. 112 (4) (1986) 20564, [http://dx.doi.org/10.1061/\(ASCE\)0733-9445\(1986\)112:4\(937\)](http://dx.doi.org/10.1061/(ASCE)0733-9445(1986)112:4(937)).
- [5] T.J.R. Hughes, J.A. Evans, A. Reali, Finite element and NURBS approximations of eigenvalue, boundary-value, and initial-value problems, Comput. Methods Appl. Mech. Engrg. 272 (2014) 290–320, <http://dx.doi.org/10.1016/j.cma.2013.11.012>.
- [6] R.H. Lopez, C.R.A. Silva Jr., A non-intrusive methodology for the representation of crack growth stochastic processes, Mech. Res. Commun. 64 (2015) 23–28, <http://dx.doi.org/10.1016/j.mechrescom.2014.12.005>.
- [7] D. Colmenares, A. Andersson, R. Karoumi, Closed-form solution for mode superposition analysis of continuous beams on flexible supports under moving harmonic loads, J. Sound Vib. 520 (2022) 116587, <http://dx.doi.org/10.1016/j.jsv.2021.116587>.
- [8] R. Cortell, Application of the fourth-order Runge–Kutta method for the solution of high-order general initial value problems, Comput. Struct. 49 (3) (1993) 897–900, [http://dx.doi.org/10.1016/0045-7949\(93\)90036-D](http://dx.doi.org/10.1016/0045-7949(93)90036-D).
- [9] T.E. Simos, J.V. Aguiar, A modified phase-fitted Runge–Kutta method for the numerical solution of the Schrödinger equation, J. Math. Chem. 30 (2001) 121–131, <http://dx.doi.org/10.1023/A:1013185619370>.
- [10] J.H. Verner, Numerically optimal Runge–Kutta pairs with interpolants, Numer. Algorithms 53 (2010) 383–396, <http://dx.doi.org/10.1007/s11075-009-9290-3>.
- [11] T.E. Simos, Ch Tsitouras, Evolutionary derivation of Runge–Kutta pairs for addressing inhomogeneous linear problems, Numer. Algorithms 87 (2021) 511–525, <http://dx.doi.org/10.1007/s11075-020-00976-9>.
- [12] C. Runge, Ueber die numerische Auflösung von Differentialgleichungen, Math. Ann. 46 (1895) 167–178, <http://dx.doi.org/10.1007/BF01446807>.
- [13] M. Kutta, Beitrag zur nherungsweise Integration totaler Differentialgleichungen, Z. Math. Phys. 46 (1901) 435–453.
- [14] J.C. Butcher, Numerical methods for ordinary differential equations in the 20th century, J. Comput. Appl. Math. 125 (1) (2000) [http://dx.doi.org/10.1016/S0377-0427\(00\)00455-6](http://dx.doi.org/10.1016/S0377-0427(00)00455-6).
- [15] J. Vigo-Aguiar, H. Ramos, A family of A-stable Runge–Kutta collocation methods of higher order for initial-value problems, IMA J. Numer. Anal. 27 (4) (2007) 798–817, <http://dx.doi.org/10.1093/imanum/drl040>.
- [16] J.M. Aristoff, J.T. Horwood, A.B. Poore, Implicit-Runge–Kutta-based methods for fast, precise, and scalable uncertainty propagation, Celestial Mech. Dynam. Astronom. 122 (2015) 169–182, <http://dx.doi.org/10.1007/s10569-015-9614-7>.
- [17] Ch Tsitouras, Ith Famelis, R.E. Simos, Phase-fitted Runge–Kutta pairs of orders 8(7), J. Comput. Appl. Math. 321 (2017) 226–231, <http://dx.doi.org/10.1016/j.cam.2017.02.030>.
- [18] T. Matsuda, Y. Miyatake, Generalization of partitioned Runge–Kutta methods for adjoint systems, J. Comput. Appl. Math. 388 (2021) 113308, <http://dx.doi.org/10.1016/j.cam.2020.113308>.
- [19] L. Zheng, X. Zhang, Modeling and Analysis of Modern Fluid Problems, Academic Press, 2017, <http://dx.doi.org/10.1016/C2016-0-01480-8>.
- [20] C. Amann, K. Kadau, Numerically efficient modified Runge–Kutta solver for fatigue crack growth analysis, Eng. Fract. Mech. 161 (2016) 55–62, <http://dx.doi.org/10.1016/j.engfracmech.2016.03.021>.
- [21] H. Millwater, J. Ocampo, N. Crosby, Probabilistic methods for risk assessment of airframe digital twin structures, Eng. Fract. Mech. 221 (2019) 106674, <http://dx.doi.org/10.1016/j.engfracmech.2019.106674>.
- [22] Y.-S. Shih, G.-Y. Wu, Effect of vibration on fatigue crack growth of an edge crack for a rectangular plate, Int. J. Fatigue 24 (5) (2002) 557–566, [http://dx.doi.org/10.1016/S0142-1123\(01\)00110-4](http://dx.doi.org/10.1016/S0142-1123(01)00110-4).
- [23] C.J. Li, H. Lee, Gear fatigue crack prognosis using embedded model gear dynamic model and fracture mechanics, Mech. Syst. Signal Process. 19 (4) (2005) 836–846, <http://dx.doi.org/10.1016/j.ymssp.2004.06.007>.
- [24] J. Xu, M.Y. Zhu, B.H. Liu, Y.T. Sun, Y.B. Li, A new vehicle speed estimation method based on the longest radial crack on windshield, Appl. Mech. Mater. 34–35 (2010) 512–516, <http://dx.doi.org/10.4028/www.scientific.net/amm.34-35.512>.
- [25] Y. Chen, R. Zhu, G. Jin, Y. Xiong, Influence of crack depth on dynamic characteristics of spur gear system, J. Vib. Eng. Technol. 7 (2019) 227–233, <http://dx.doi.org/10.1007/s42417-019-00115-2>.
- [26] Y.-S. Shih, Y.-S. Wang, Vibration and fatigue crack growth of a ferromagnetic and rectangular cracked plate subjected to a transverse magnetic field, Eng. Fract. Mech. 259 (2022) 108146, <http://dx.doi.org/10.1016/j.engfracmech.2021.108146>.
- [27] A.T. Beck, R.E. Melchers, Overload failure of structural components under random crack propagation and loading - a random process approach -, Struct. Saf. 26 (2004) 471–488, <http://dx.doi.org/10.1016/j.strusafe.2004.02.001>.
- [28] L. Dieci, L. Lopez, A survey of numerical methods for IVPs of ODEs with discontinuous right-hand side, J. Comput. Appl. Math. 236 (16) (2012) 3967–3991, <http://dx.doi.org/10.1016/j.cam.2012.02.011>.

- [29] W.H. Enright, K.R. Jackson, S.P. Norsett, P.G. Thomsen, Interpolants for Runge–Kutta formulas, *ACM Trans. Math. Software* 12 (3) (1986) 193–218, <http://dx.doi.org/10.1145/7921.7923>.
- [30] M. Zennaro, Natural continuous extensions of Runge–Kutta methods, *Math. Comp.* 46 (173) (1986) 119–133, <http://dx.doi.org/10.2307/2008218>.
- [31] L.F. Shampine, Jay L.O., Dense output, in: Bjorn Engquist (Ed.), *Encyclopedia of Applied and Computational Mathematics*, Springer Berlin Heidelberg, 2015, pp. 339–345, http://dx.doi.org/10.1007/978-3-540-70529-1_107.
- [32] D.I. Ketcheson, L. Lóczi, A. Jangabylova, A. Kusmanov, Dense output for strong stability preserving Runge–Kutta methods, *J. Sci. Comput.* 71 (2017) 944–958, <http://dx.doi.org/10.1007/s10915-016-0331-5>.
- [33] R. Mannshardt, One-step methods of any order for ordinary differential equations with discontinuous right-hand sides, *Numer. Math.* 31 (1978) 131–152, <http://dx.doi.org/10.1007/BF01397472>.
- [34] M. Calvo, J.I. Montijano, L. Rández, The numerical solution of discontinuous IVPs by runge–kutta codes: A review, *SeMA J. Boletín Sociedad Española Matemática Aplicada* 44 (2008) 33–54.
- [35] J. Stoer, R. Bulirsch, *Introduction to Numerical Analysis*, second ed., Springer-Verlag, New York, 1993.
- [36] C.R.A. Silva Jr., R.V. Santos, A.T. Beck, Analytical bounds for efficient crack growth computation, *Appl. Math. Model.* 40 (2016) 2312–2321, <http://dx.doi.org/10.1016/j.apm.2015.09.053>.
- [37] K. Sobczyk, B.F. Spencer, *Random Fatigue: From Data to Theory*, Academic Press, London, 1992.
- [38] H.O. Madsen, Stochastic modeling of fatigue crack growth and inspection, in: C. Guedes Soares (Ed.), *Probabilistic Methods for Structural Design*, Springer, Dordrecht, 1997, pp. 59–83, http://dx.doi.org/10.1007/978-94-011-5614-1_4.
- [39] T.L. Anderson, *Fracture Mechanics: Fundamentals and Applications*, third ed., Taylor & Francis Group, Boca Raton, 2005, <http://dx.doi.org/10.1201/9781420058215>.
- [40] A.T. Beck, W.J.S. Gomes, Stochastic fracture mechanics using polynomial chaos, *Probab. Eng. Mech.* 34 (2013) 26–39, <http://dx.doi.org/10.1016/j.probenmech.2013.04.002>.
- [41] W.J.S. Gomes, A.T. Beck, Optimal inspection planning and repair under random crack propagation, *Eng. Struct.* 69 (2014) 285–296, <http://dx.doi.org/10.1016/j.engstruct.2014.03.021>.
- [42] P.C. Paris, F. Erdogan, A critical analysis of crack propagation laws, *J. Basic Eng.* 85 (4) (1963) 528–534, <http://dx.doi.org/10.1115/1.3656900>.
- [43] J.C. Newman Jr., I.S. Raju, An empirical stress-intensity factor equation for the surface crack, *Eng. Fract. Mech.* 15 (1–2) (1981) 185–192, [http://dx.doi.org/10.1016/0013-7944\(81\)90116-8](http://dx.doi.org/10.1016/0013-7944(81)90116-8).
- [44] B. Mechab, B. Serier, B. Bachir Bouiadjra, K. Kaddouri, X. Feaugas, Linear and non-linear analyses for semi-elliptical surface cracks in pipes under bending, *Int. J. Press. Vessels Pip.* 88 (1) (2011) 57–63, <http://dx.doi.org/10.1016/j.ijpvp.2010.11.001>.
- [45] J. Predan, V. Mocilnik, N. Gubelj, Stress-intensity factors for circumferential semi-elliptical surface cracks in a hollow cylinder subjected to pure torsion, *Eng. Fract. Mech.* 105 (2013) 152–168, <http://dx.doi.org/10.1016/j.engfracmech.2013.03.033>.
- [46] K. Walker, The effect of stress ratio during crack propagation and fatigue for 2024-T3 and 7075-T6 aluminum, in: ASTM STP 462, American Society for Testing and Materials, Philadelphia, PA, 1970, pp. 1–14.
- [47] American Petroleum Institute, API 579-1/ASME FFS-1: Fitness-for-service, 2016.
- [48] J. Schijve, *Fatigue of Structures and Materials*, second ed., Springer, Netherlands, 2009, <http://dx.doi.org/10.1007/978-1-4020-6808-9>.
- [49] P.L. DeVries, *A First Course in Computational Physics*, John Wiley & Sons, Canada, 1994.
- [50] P.G. O'Regan, Step size adjustment at discontinuities for fourth order Runge–Kutta methods, *Comput. J.* 13 (4) (1970) 401–404, <http://dx.doi.org/10.1093/comjnl/13.4.401>.
- [51] L.F. Shampine, M.W. Reichelt, The MATLAB ODE suite, *SIAM J. Sci. Comput.* 18 (1997) 1–22, <http://dx.doi.org/10.1137/S1064827594276424>.
- [52] P. Bogacki, L.F. Shampine, An efficient Runge–Kutta (4, 5) pair, *Comput. Math. Appl.* 32 (6) (1996) 15–28, [http://dx.doi.org/10.1016/0898-1221\(96\)00141-1](http://dx.doi.org/10.1016/0898-1221(96)00141-1).
- [53] R. Archibald, A. Gelb, J. Yoon, Determining the locations and discontinuities in the derivatives of functions, *Appl. Numer. Math.* 58 (2008) 577–592, <http://dx.doi.org/10.1016/j.apnum.2007.01.018>.
- [54] M.B. Carver, Efficient implementation over discontinuities in ordinary differential equation simulations, *Math. Comput. Simulation* 20 (3) (1978) 190–196, [http://dx.doi.org/10.1016/0378-4754\(78\)90068-X](http://dx.doi.org/10.1016/0378-4754(78)90068-X).
- [55] J.R. Cash, A.H. Karp, A variable order Runge–Kutta method for initial value problems with rapidly varying right-hand sides, *ACM Trans. Math. Software* 16 (1990) 201–222, <http://dx.doi.org/10.1145/79505.79507>.
- [56] P. Bogacki, L.F. Shampine, A 3(2) pair of Runge–Kutta formulas, *Appl. Math. Lett.* 2 (1989) 321–325, [http://dx.doi.org/10.1016/0893-9659\(89\)90079-7](http://dx.doi.org/10.1016/0893-9659(89)90079-7).
- [57] D.J. Higham, L.N. Trefethen, Stiffness of ODEs, *BIT Numer. Math.* 33 (1993) 285–303, <http://dx.doi.org/10.1007/BF01989751>.
- [58] J.R. Dormand, P.J. Prince, A family of embedded Runge–Kutta formulae, *J. Comput. Appl. Math.* 6 (1980) 19–26, [http://dx.doi.org/10.1016/0771-050X\(80\)90013-3](http://dx.doi.org/10.1016/0771-050X(80)90013-3).
- [59] M. Deserno, *How to Generate Exponentially Correlated Gaussian Random Numbers*, Department of Chemistry and Biochemistry, UCLA, USA, 2002.
- [60] J.A. Bannantine, J.J. Comer, J.L. Handrock, *Fundamentals of Metal Fatigue Analysis*, Prentice Hall, New Jersey, 1989.
- [61] E.E. Gdoutos, *Fracture Mechanics: An Introduction*, third ed., Springer, 2020, <http://dx.doi.org/10.1007/978-3-030-35098-7>.
- [62] W.J.S. Gomes, A.G. Garbisi, A.T. Beck, Hybrid MCS-FORM approach to solve inverse fracture mechanics reliability problems, *Struct. Multidiscip. Optim.* 65 (2022) 77, <http://dx.doi.org/10.1007/s00158-022-03182-4>.